
cTreeBalls

Release 1.0.0

Mario A. Rodriguez-Meza

Mar 06, 2026

CONTENTS:

1	Overview	1
1.1	Introduction	1
1.2	Compiling and getting started	1
1.3	Configuration	2
1.4	Parameters	2
1.5	Python	2
1.6	Plotting utilities	2
1.7	License	3
1.8	Acknowledgements	3
2	Installation	5
2.1	Makefile	5
2.2	Makefile settings	5
2.3	Makefile plataform machine	5
3	Parameters	7
3.1	Parameters related to the searching method	7
3.2	Parameters to control the I/O file(s)	7
3.3	Parameters to control histograms and their output files	8
3.4	Set of parameters needed to construct a test model	8
3.5	Miscellaneous parameters	8
4	Linear power spectrum files	11
4.1	Standard formats	11
5	Pre/Post processing	13
5.1	Pre-processing	13
5.2	Post-processing	13
6	AddOn's	15
6.1	Include files	15
6.2	Source files	15
7	Python interface	17
7.1	Using command line interface	17
7.2	Cython interface	17
8	Hands on	19
9	2-point correlation functions	21

10 3-point correlation functions	23
10.1 Harmonic base	23
11 Indices and tables	25

OVERVIEW

cTreeBalls: Correlation functions computation with Tree/Balls methods

Authors: A. Aviles, J.C. Hidalgo, E.A. Moreno-Alcala, G. Niz, S. Ramirez, Mario A. Rodriguez-Meza, S. Samario-Nava

For download and information, see <https://github.com/rodriguezmeza/wlcf.git>

1.1 Introduction

Correlation function computation with Tree/Balls methods (short-name cBalls) is a C code for computing correlation functions using tree and balls methods. So far can compute 2 points correlation function (2pcf) and 3 points correlation function for scalar fields like weak lensing convergence.

1.2 Compiling and getting started

Download the code by cloning it from <https://github.com/rodriguezmeza/cTreeBalls>

Dependencies: cBalls optionally needs gsl version 2.7.1 and cfitsio version 4.4.1 installed in your system. Go to its web page [GSL](#) for details or ask to your system administrator. Make necessary changes in `Makefile_machine` file and look up for GSL. Can be switched off: `USEGSL = 0` in `Makefile_settings`. I/O cfitsio library is set it OFF. Can be set it ON in `addons/Makefile_settings`.

Go to the cTreeBalls directory (`cd cTreeBalls`) and compile (`make clean; make`). If the first compilation attempt fails, you may need to open the `Makefile_machine` file and adapt the name of the compiler (default: `gcc`), of the optimization flag (default: `-O4 -ffast-math`) and of the OpenMP flag (default: `-fopenmp`; this flag is facultative, you are free to compile without OpenMP if you don't want parallel execution; note that you need the version 4.2 or higher of `gcc` to be able to compile with `-fopenmp`). The code has been tested with `gcc` version 10 and would be working with version 11, 12. (In particular, for compiling on Mac ≥ 10.9 despite of the clang incompatibility with OpenMP).

To check that the code runs, type:

```
cd tests
../cballs parameters_explained
```

The `parameters_explained` file is a reference input file, containing and explaining the use of all possible input parameters.

By default **cBalls** reads/writes catalog of points to analyzed files in 4-column format with `x`, `y`, `z` columns first and then value of the convergence field. It has a two line header:

```
# nbody NDIM Lx Ly Lz
# nbody-value NDIM-value Lx Ly Lz - values
```

Try running:

```
../wlcfc
```

In the Output directory you will have a file: `points_on_sphere.txt`. View its contents to see the two lines header and the 4 columns structure of data.

Note: in the above example `points_on_sphere` was not given an extension. By default cBalls gives to the output files the extension `.txt`.

On top of that, if you wish to modify the code, you will find comments directly in the files in the folder `addons`, and the modules you may add must go in this folder. See one of the most simple in `addons/direct_method`.

For the moment you may consult man page:

```
$ man ../docs/wlcfc.1
```

or open with a browser the html file: `docs/man/wlcfc.html`

See more details about parameters needed by cBalls below ([Parameters](#)).

1.3 Configuration

wlcfc can be configured by switching on/off several options. Configuration file is `Makefile_setting`.

1.4 Parameters

The list of available command line parameters can be consulted using the `--help` flag:

```
../wlcfc --help
```

or its short version:

```
../wlcfc -h
```

1.5 Python

To install wlcfc python module (`wlcfcpy`) just execute:

```
$ make clean; make all
```

To test it go to directory `tests` and 'run':

```
python python/Bell.py
```

Note: this interface in Cython was tested in a python environment with `python3.13`.

1.6 Plotting utilities

Several Python scripts, written by Mario A. Rodriguez-Meza, are available to process wlcfc results. They are here in the same github repository in the `tests/python` folder

1.7 License

wlcf is written by A. Aviles et al., and is distributed under the [MIT license](#). If you use this program in research work that results in publications, please cite the following paper:

Abraham Arvizu et al., [arXiv:2048.16847](#)

1.8 Acknowledgements

wlcf use/is based on the following codes or projects:

- [FFTLog](#)
- [The BiHaloFit model of Takahashi](#)
- [Zeno](#)
- [Numerical recipes](#)
- [GSL](#)
- [CLASS](#)

INSTALLATION

This section describes the installation settings of cBalls:

2.1 Makefile

2.2 Makefile settings

2.3 Makefile plataform machine

PARAMETERS

This section describes the various parameters cBalls needs for controlling what the searching process do:

3.1 Parameters related to the searching method

searchMethod

(str or list) [alias: search] The searching method to use. Default is `tree-omp-sincos`. Fastest method so far is `balls-omp`.

Use it as:

```
searchMethod = balls-omp
```

In command line version do not use spaces before and after = or it won't be parsed correctly. In a parameter file you have more liberty.

mChebyshev

(positive int) [alias: mcheb] The number of multipoles to compute for a 3pcf computation. Number of them are: `mChebyshev + 1`, because it includes the monopole.

Use it as:

```
mChebyshev = 7
```

Default value is 7. As multipoles comes from a harmonic expansion, we may be interested in computing the whole 3pcf. This process involves a FFT, therefore give `mChebyshev + 1` as a power of 2.

3.2 Parameters to control the I/O file(s)

infile

(str, default='') [alias: in] File names with points to analyse.

infileformat

(str, default=columns-ascii) [alias: infmt] Data input files format (columns-ascii, binary or takahasi)

The input columns for `columns-ascii` format:

- `x` = x position of a point in the catalog.
- `y` = y position of a point in the catalog.
- `z` = z position of a point in the catalog.
- `kappa` = The kappa value of the point.

3.3 Parameters to control histograms and their output files

useLogHist

(bool, default=true) Which type of binning should be used.

logHistBinsPD

(float) The minimum separation to include in the histograms.

rangeN

(float) The maximum separation to include in the histograms.

rminHist

(float) The minimum separation to include in the histograms.

sizeHistN

(int) The number of output bins to use.

3.4 Set of parameters needed to construct a test model

seed

(int, default=123) Random number seed to test run or useful to change a random region in Takahasi simulations.

testmodel

(str, default=simple-cubic-random) [alias: tstmodel] Test model name to analyse.

nbody

(int, default=16348) Number of points to test.

lengthBox

(float, default=10000) [alias: lbox] Length of the box to test.

3.5 Miscellaneous parameters

script

(str, default=``) Scripts in shell or python that can be run in pre-processing or post-processing.

stepState

(int, default=10000) number of steps to save a state-run info (pivot number completed in the log file).

verbose

(int, default=1) [alias: verb] How verbose the code should be during processing.

- 0 = no output unless there is an error
- 1 = output warnings
- 2 = output progress information
- 3 = output extra debugging lines

verbose_log

(int, default=1) [alias: verblog] To print messages to a log file `cballs.log` in directory `tmp` under output directory given by the parameter: `rootDir`.`

Amount of message information is controlled by the int given.

numberThreads

(int, default=4) [alias: nthreads] How many OpenMP threads should be used.

It is needed to switch on OpenMP: `OPENMP_MACHINE = 1` in `Makefile_settings` and recompile cBalls again.

options

(str, default="") [alias: opt] You may give here various code behavior options.

Use it as:

```
options = str1,str2,str3,...
```

where str# is one of the:

- stop = stop execution before searching process
- compute-HistN = compute NN encounters and save histogram in a file
- and-CF = if you use `compute-HistN` then you may compute and save the correlation function of NN encounters (the equivalent to the radial distribution function in liquids).
- no-one-ball = during the searching process does not use balls criterion to speed up the code

Note

- It is not necessary to specify all the parameters. You need to give only the ones appropriate to the run. The rest of parameters will use their default values if they are OK with you.
- When you specify the root output directory using: `rootDir`, and this is a single directory that will be located in the pwd dir, then do not use `./` at the beginning of the name or `/` at its end.

LINEAR POWER SPECTRUM FILES

This section describes format of linear power spectrum you need to compute correlation functions using `wlcf`.

4.1 Standard formats

columns-ascii

The file(s) with the linear power spectrum data.

PRE/POST PROCESSING

This section describes the various pre/pos processing for controlling what the *cballs* before/after main processing:

5.1 Pre-processing

5.2 Post-processing

ADDON'S

This section describes the mechanism to add more functionally to cBalls:

6.1 Include files

6.2 Source files

PYTHON INTERFACE

This section describes the mechanism to add more functionally to cBalls:

7.1 Using command line interface

7.2 Cython interface

HANDS ON

This section describes a practical guide to use **cBalls**:

Let us test one of the [Takahasi](#) realizations. We download the realization using unix `wget` command in the terminal:

```
wget http://cosmo.phys.hirosaki-u.ac.jp/takahasi/allsky_raytracing/sub1/nres12/allskymap_  
↪nres12r000.zs9.mag.dat
```

Then we will have in our working directory the file `allskymap_nres12r000.zs9.mag.dat`, 3 Gb in size with ~200 million points distributed on the surface of a unit sphere.

2-POINT CORRELATION FUNCTIONS

cBalls can compute 2-point correlations (2pcf) when counts (N) or a scalar field are involved (like convergence K):

NN

It is the normal 2-point correlation function of number counts (typically galaxy counts).

KK

So far we have consider that the scalar field is the convergence in weak lensing κ . Then this gives the 2-point kappa-kappa correlation function.

N represent simple counting and K represent a real scalar field, like convergence in weak lensing.

3-POINT CORRELATION FUNCTIONS

cBalls can also compute 3-point correlations (3pcf) when counts (N) or a scalar field are involved (like convergence κ). In particular to compute the correlation **KKK** the numerical code use the harmonic base.

10.1 Harmonic base

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`